

1. The 8-puzzle game is a deterministic environment controlled by a single agent. The goal is to find a path to a winning state in the smallest amount of moves possible. The Tic-Tac-Toe game is a competitive and strategic environment with 2 agents making moves. The goal is to find a state in which the Max player has the best possible score while being undermined by the Min player.

The main difference between the 2 are their goals. The 8-puzzle attempts to find a specific, unchanging sequence of index positions as the goal. This means that its wants it's goal state to look like [1,2,3,4,5,6,7,8,0] and will perform movements to reach that goal state. It also always has a path to the goal state as there are no limitations or bounds that would end the game besides reaching the goal state or quitting the game. However, the Tic-Tac-Toe game has a specific line sequence (such as 3 X's in a row) as its goal state and can be achieved in various ways, for example horizontally or diagonally. There are also multiple terminal states that have different values to each other. For example, a victory for the X player would not have the same value as a tie.

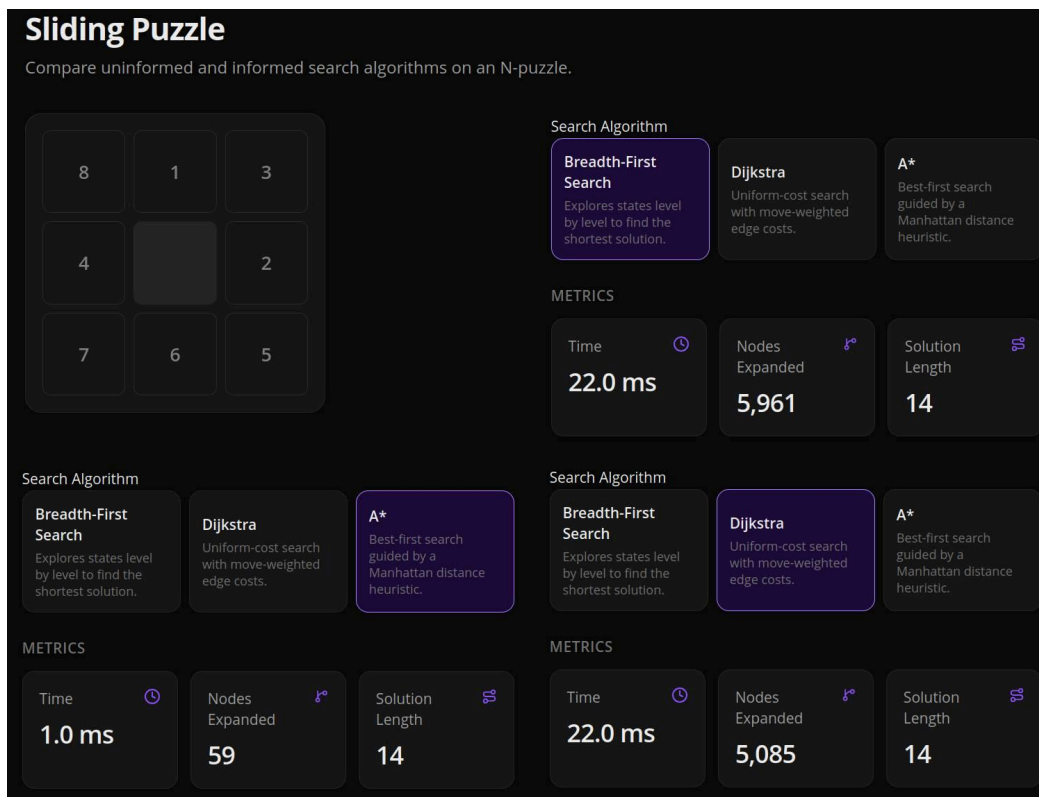
2. The A* algorithm is appropriate for the 8-puzzle but not to Tic-Tac-Toe the former has a direct and unchanging goal state while the latter has to compete against another agent making decisions, which can have various different terminal states. Furthermore, Tic-Tac-Toe cares about the value of the terminal state (win, draw, or loss) and not how to most effectively reach those states. The A* algorithm is designed to find the least-cost path and, as such, is more suited to the 8-Puzzle.

3.

```
Bruhdy@DESKTOP-B4T1FKR MINGW64 ~/github/ai-search-lab (main)
$ npx tsx --tsconfig tsconfig.app.json scripts/BenchmarkPuzzle.ts
Puzzle benchmark - standardized test puzzle

Start: [8, 1, 3, 4, 0, 2, 7, 6, 5]
Goal:  [1, 2, 3, 4, 5, 6, 7, 8, 0]

bfs      | nodes: 5961 | moves: 14 | time: 20.8ms
dijkstra | nodes: 5085 | moves: 14 | time: 18.5ms
astar    | nodes: 59   | moves: 14 | time: 0.4ms
```

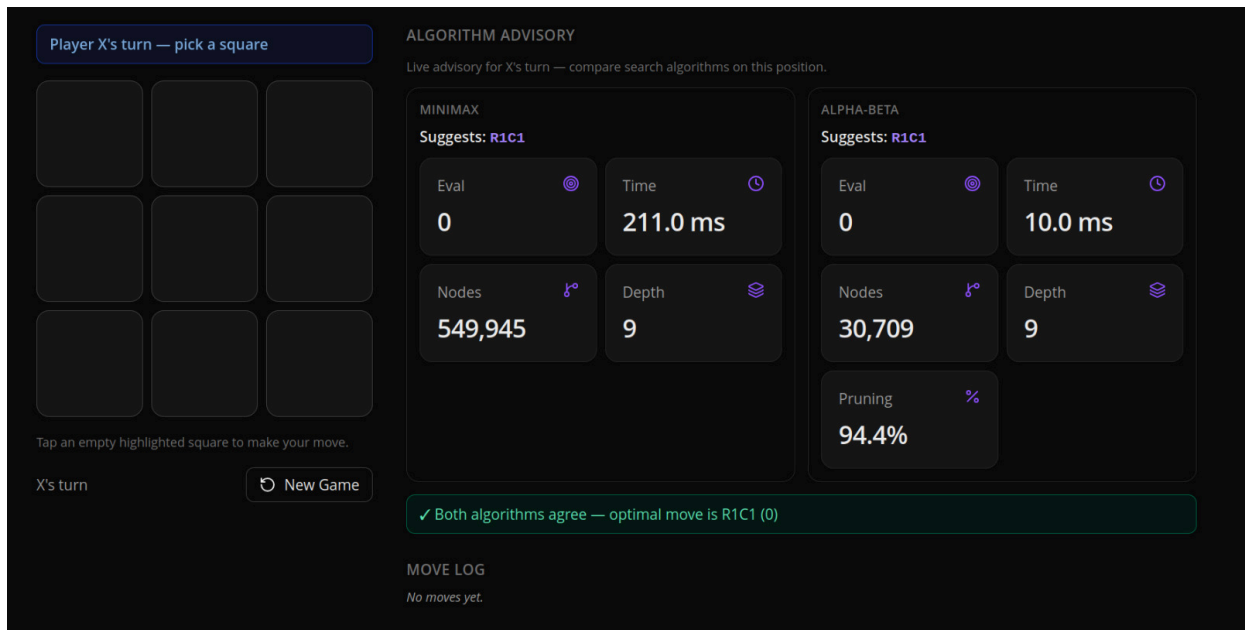


With the nodes expanded for blind search, BFS had the most, followed by dijkstra, and then A*. This is because each one had varying levels of heuristic search involved. BFS has no heuristic involved, and instead finds the shortest path. However, it takes the longest as it decides to search without any guidance, and instead goes level by level, which ends up taking 5961 nodes to find. Dijkstra's heuristic finds the least-cost path by tracking the cost to each node and updating it until it finds the cheapest path to the goal, which ends up taking 5085 nodes. A* has a more advanced heuristic that combines Dijkstra's with Manhattan Distance. This means that it finds the cheapest path, while being pulled in the direction of the goal state, and as such only needs 59 nodes explored to reach the goal state. This tells us that heuristics create an informed search which allows for a quicker way to find a path to the goal state, however it does increase complexity.

4.

```
Bruhdy@DESKTOP-B4T1FKR MINGW64 ~/github/ai-search-lab (main)
$ npx tsx --tsconfig tsconfig.app.json scripts/BenchmarkTicTacToe.ts
Tic-Tac-Toe benchmark - empty board, X plays first

Minimax      | Nodes: 549945 | Depth: 9 | Score: 0 | time: 95.4 ms |
Alpha-Beta   | Nodes: 30709  | Depth: 9 | Score: 0 | time: 7.4 ms | Nodes Pruned: 519236 |
Pruning Rate: 0.9441598705325078
```



(Note: Pruning Rate is part of the Alpha-Beta Algorithm and is after the Nodes Pruned)

The Minimax algorithm searches all nodes to the terminal state, and attempts to maximize its score against a minimizing player. The Max player chooses the max score of its children nodes and the Min player chooses the min score of its children nodes. They alternate turns as to simulate a competitive multi-agent environment. As such, it takes a long time to search through all 549,945 nodes. Alpha-Beta Pruning has a similar algorithm to min-max, except it stores a global Max and Min value that is guaranteed (Alpha and Beta respectively). This process is called pruning and allows the algorithm to skip any nodes that would not affect the final decision between the Min and Max players. As such, it has a significantly less amount of nodes explored, because there are other nodes that are already guaranteed to give a higher score so the lower score nodes are skipped. The Alpha-Beta Pruning only visits 30,709 nodes and prunes 519,236. The total of these 2 is 549,945 which is the amount of nodes visited by the normal Minimax. The Pruning Rate is calculated by taking the number of Alpha-Beta nodes and dividing it by the number of nodes the normal minimax algorithm would reach, which turns out to be approximately 94%.

5.

Completeness:

- BFS is complete as it guarantees to find a path to the goal state if one exists by searching all nodes level by level.
- Dijkstra is complete as it guarantees to find a path to the goal state if one exists by searching all the least-cost paths to each node.

- c. A* is complete as it guarantees to find a path to the goal state if one exists by choosing the least-cost path with another heuristic (ours used Manhattan Distance) as well.
- d. Minimax is complete as it guarantees to find a path to a goal state if one exists by searching all nodes to their terminal states.
- e. Alpha-Beta Pruning is complete as it guarantees to find a path to a goal state if one exists by searching all necessary nodes to their terminal states and only skipping those that wouldn't affect the final decision.

Optimality

- a. BFS is only optimal if each edge has the same cost, as it finds the shortest path.
- b. Dijkstra is optimal as it finds the least-cost path to the goal state.
- c. A* is optimal as it finds the least-cost path to the goal state using a heuristic (Manhattan Distance).
- d. Minimax is only optimal if the opponent plays optimally, as it assumes and plays in a Best and Worst case fashion. This means that it skips certain nodes as it assumes the worst, when the opponent might not choose the worst.
- e. Alpha-Beta Pruning is optimal as it chooses the best moves and skips debating any moves that aren't as valuable.

Time Complexity

- a. BFS has a time complexity of $O(|V| + |E|)$ where V is the number of nodes and E is the number of edges.
- b. Dijkstra has a time complexity of $O((|V| + |E|) \log |V|)$ where V is the number of nodes and E is the number of edges.
- c. A* has a time complexity of $O(b^m)$ where b is the branching factor and m is the max depth possible.
- d. Minimax has a time complexity of $O(b^m)$ where b is the branching factor and m is the max depth possible.
- e. Alpha-Beta Pruning has an average time complexity of $O(b^{m/2})$ where b is the branching factor and m is the max depth possible. However, its worst case is $O(b^m)$.

Space Complexity

- f. BFS has a space complexity of $O(|V|)$ where V is the number of nodes.
- g. Dijkstra has a space complexity of $O(|V|)$ where V is the number of nodes.
- h. A* has a space complexity of $O(b^m)$ where b is the branching factor and m is the max depth possible.
- i. Minimax has a space complexity of $O(b * m)$ where b is the branching factor and m is the max depth possible.

- j. Alpha-Beta Pruning has a space complexity of $O(b * m)$ where b is the branching factor and m is the max depth possible.

Heuristic Justification

We chose the Manhattan distance as our heuristic. For each tile, it sums the minimum row steps and column steps to its goal position. Since only one tile moves one square per turn, the total Manhattan distance can decrease by at most 1 per move, so the true solution length can never be shorter than that sum. Therefore the heuristic never overestimates